

Seq2Image: Sequence Analysis using Visualization and Deep Convolutional Neural Network

Neda Tavakoli

College of Computing, School of Computer Science
Georgia Institute of Technology, Atlanta, GA, USA
Email: neda.tavakoli@gatech.edu

Abstract—Sequence classification has been widely used in numerous application domains. There exists a good number of classification algorithms that can be applied to feature vectors. However, these classification algorithms cannot be directly applied to the sequence classification problem, mainly because of the difficulties to capture feature vectors from sequences. More specifically, due to the sequential nature of features that exist in a sequence, the clustering problem in sequences suffers from the curse of dimensionality, which makes the sequence classification task more challenging compared to a typical classification on feature vectors. In this paper, we present a novel idea of transforming sequences to images, called Seq2Image, a simple yet effective method to perform genomic sequence classification using Convolutional Neural Network (CNN). We first convert a given genomic sequence to a tensor, and then the obtained tensor is transformed into an image. We then employ the CNN deep learning-based image processing techniques to classify the created images of sequences. The results of our preliminary experimental study are very promising achieving 95.78% training accuracy, 95.76% validation accuracy, and 95.83% testing accuracy for classification of human genome of 166 samples with six different sequence families.

Index Terms—Seq2Image, Sequence Classification, Convolutional Neural Network

I. INTRODUCTION

Sequence classification has been widely used in numerous application domains such as genomic analysis and protein classification, financial time series analysis and clustering, information retrieval, document classification, and intrusion detection. In the field of genomic analysis, sequence classification is used to classify protein sequences into certain groups and then learn the functionalities of new proteins [1]. Sequence classification is also used in health informatics to classify the time series data of heart rates (called Electrocardiogram or ECG) in order to examine if a person is healthy or not [2]. Sequence classification is also used in intrusion detection to perform anomaly detection [3]. Moreover, in the field of information retrieval, sequence classification is applied to classify documents into different topics [4].

In general, sequence classification methods are grouped into three categories [5]: 1) feature-based classification in which a sequence is transformed into a feature vector in order to leverage the capabilities of conventional classification methods, 2) sequence distance-based classification methods in which the similarity between sequences are measured, and then similar sequences are grouped into the same class, and 3) model-based sequence classification approaches in which

statistical models such as Hidden Markov Models (HMM) are employed to classify sequences.

This paper presents the novel idea of transforming sequences into images, called “Seq2Image,” to analyze and cluster sequence data. The introduced approach is a feature-based classification method that leverages Convolutional Neural Networks (CNN) to classify feature vectors of genomic sequences. CNNs are usually designed to classify feature vectors and, as a result, the main challenging part of sequence classification problem is the extraction of feature vectors from sequences. This challenge is mainly due to the high dimensionality of features that exist in a sequence. To address this problem, we first transform a sequence to a tensor, which is similar to covariance matrix that represents relationship among features. The obtained tensor is then transformed into an image, called a sequence image. We used multiple mapping functions to transform a genomic sequence to an image, which is discussed in more details in the following sections. Once we obtained sequence images, in the next step, we build training, validation and testing image dataset. To do so, all the images must have labels. We apply a hierarchical clustering algorithm on the sequence images to generate labels for images. The search for the best hierarchical clustering and thus labeling is performed through the process of splitting the given data into the training, validation and testing datasets, and then performing a series of splitting accuracy checking.

In our experimentation, the training dataset contains 72% of the total labeled dataset, the validation dataset has 12% of the total dataset, which is used to test how well the built model performs against known labeled data. It is also needed to test how well the model performs against unseen labeled data. To do so, we generate new sequence images (i.e., test dataset) using the same mapping function that we used for generating training and validation image dataset. Similarly, we leverage hierarchical clustering to generate labels for the new images. As a part of an unbiased evaluation and accuracy checking, since our model has not trained on these new images, we can utilize them to evaluate our model and its accuracy and then observe how well the model can classify sequences over unseen datasets (i.e., test dataset). The test dataset contains 16% of the total labeled dataset. Once the training, validation, and testing datasets (i.e., sequence images) are built, the problem is then transformed into a conventional image-clustering problem. As a result, well-known techniques

such as the Convolutional Neural Network (CNN) modeling can be applied to classify images of sequences and thus the sequences themselves. The main advantage of using CNN in image classification is that this technique does not require features to be extracted beforehand. The CNN model is able to learn features on its own. This capability has made CNN to become one of the most effective tools to classify images with high order of accuracy. We evaluated our framework using human genome DNA sequences that are available in public databases (e.g., GenBank [6], EMBL Nucleotide Sequence Database [7]). The key contributions of this paper are:

- Introducing the novel idea of transforming sequences to images for the purpose of clustering using deep CNNs.
- Evaluating the results of clustering for various mappings.

The paper is organized as follows: Section II discusses some related work and the required backgrounds for this work. Section III discusses our proposed Seq2Image technique and our deep CNN architecture. In Section IV, the experimental results are provided. Section V concludes the paper and highlights the future directions.

II. RELATED WORK AND BACKGROUND

Deep learning models such as CNN, deep LSTM, BiLSTM are widely used in different application domains [8, 9, 10, 11]. In particular, CNN and feature-based methods are widely used for genomic and protein sequence classifications [1, 12, 13, 14]. She et al. [14] found that SVM is the most effective feature extraction method for protein classification. In general, two main concerns for sequence classification are accuracy and scalability. Accuracy indicates how precise the classification is, and scalability is the ability of handling and training a large amount of data [15, 16]. In this paper, we discuss accurate and scalable deep CNN-based classification method. Our experimental results demonstrate that the overall training accuracy of 95.78% is achieved, which is very encouraging.

A. Agglomerative Hierarchical Clustering

Agglomerative hierarchical clustering [17] is a bottom-up clustering method in which each element is initially considered as a cluster, and then based on similarity among elements, pairs of clusters are merged or split successively. A tree (or dendrogram) is used to represent the hierarchy of clusters where the root of the tree is a cluster that contains all the elements, and the leaves are the clusters containing only a single element. There exists different linkage methods to merge elements in a cluster such as: ward, complete, average, and single linkage. The ward method finds and minimizes the sum of squared differences within all clusters. This objective function is similar to that of k-mean objective function, which minimizes the variance; however, it leverages an agglomerative hierarchical method. The complete linkage method is used to minimize the maximum distance between pairs of elements in clusters. The average linkage method is used to minimize the average distance among all the pairs of elements in clusters. The single linkage method minimizes the distance between the closest pairs of elements of clusters. Agglomerative clustering follows a "rich get richer" behavior, so the single linkage

method is the worst strategy, which leads to uneven (unbalanced) cluster sizes. On the other hand, the ward method is the most efficient method, which produces most regular size clusters. However, average linkage method works better for non-Euclidean metrics because the affinity (i.e., the distance used in clustering) cannot be varied with the ward linkage method. We use the agglomerative hierarchical clustering with the ward method.

B. (Deep) Convolutional Neural Network (CNN)

Convolutional neural networks are a special form of neural networks that can be used for clustering and analyzing various types of data sets including images. These techniques take images as inputs and assign weights (and biases) to different pixels of the given image. CNNs contain two elements: convolutional layers and pooling layers. Convolutional layers are used to apply a filter to an input image in order to extract feature maps. If an image is represented (i.e., embedded) using a 2D matrix, a filter is then a smaller matrix of weights, which slides over the input 2D matrix. The filter performs element-wise multiplications (i.e., dot product) with a filter-sized patch of the input and then sums up the results to generate a single output pixel. This operation is called convolution. One of the major problems of the output feature maps is their sensitivity to the location of the features inside the input. To overcome this problem, feature maps can be down-sampled using pooling layers. More specifically, pooling layers perform down-sampling by summarizing the features inside the patches of the feature map (i.e., filter-sized sections of the image). Two commonly used pooling methods are max-pooling and average-pooling in which the most activated features and the average presence of a feature summarize respectively.

III. SEQ2IMAGE FOR SEQUENCES CLUSTERING

This section explains our proposed model to perform genome sequence classification. To construct the model, the following steps are required:

- 1) Sequence to tensor encoding: convert sequences to their associated tensors.
- 2) Sequence tensor labeling: apply agglomerative hierarchical clustering on the sequence tensors to make them labeled.
- 3) Tensor to image transformation: convert tensor to their associated images using mapping functions.
- 4) Building dataset: build and split the dataset to train, validation, and test.
- 5) Adapting convolutional neural network model to perform image classification.

Now let's explain the above steps in more details.

1) Sequence to Tensor Encoding

A sequence in our model is a set of characters. In this work, we are specifically dealing with DNA sequences, which are composed of four nucleotide characters: A, C, T, and G. For example "ACCTTGGGTAAGAGC" represents a DNA sequence. In our model, we first construct a tensor (i.e., a matrix representing the sequence) for each sequence. To do

TABLE I: An example of a tensor sequence.

Given Sequence									
GATCACAGGTCTATCACCTATTAACCACTCAC									
GGGAGCTCTCCATGCATTTGGTATTTTCGTCTG									
GGGGGTATGCACGCGATAGCATTGCGAGACGCTG									
Tensor Sequence									
71	65	84	67	65	67	65	71	71	84
67	84	65	84	67	65	67	67	67	84
65	84	84	65	65	67	67	65	67	84
67	65	67	71	71	71	65	71	67	84
67	84	67	67	65	84	71	67	65	84
84	84	71	71	84	65	84	84	84	84
67	71	84	67	84	71	71	71	71	71
71	84	65	84	71	67	65	67	71	67
71	65	84	65	71	67	65	84	84	71
67	71	65	71	65	67	71	0	0	0

so, each character is represented by its ASCII code. Given the four characters for this type of sequence data, then the ASCII code for A, C, T, and G are 65, 67, 71, and 84, respectively. The tensor matrix then can be represented by a 2D matrix. For example, Table I represents the generated tensor for the given sequence. Note that this encoding mechanism can be applied on any sequence data type.

2) Sequence Tensors Labeling

In this step, the obtained sequence tensors are labeled using agglomerative hierarchical clustering that was explained in Section II-A. This clustering is used to assign labels to each tensor. In this paper, we consider 6 labels to group sequences. We use 6 clusters mainly because in our experiments it leads to a better silhouette score. Figure 1 demonstrates the 2D and 3D results of the agglomerative hierarchical clustering on 166 human DNA sequences of size 100 that are clustered in 6 groups using hierarchical agglomerative clustering.

3) Tensor to Image Transformation

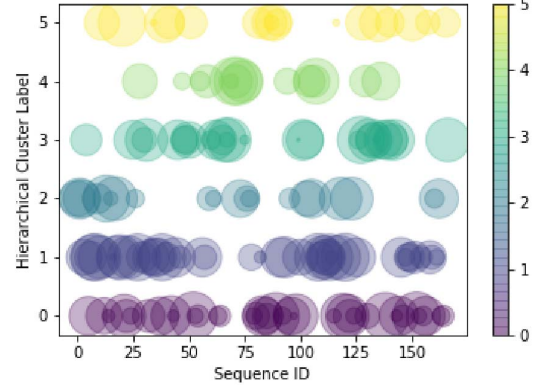
Once the tensors for the given sequences are built, we use a mapping function to convert each element of the tensor to a pixel for its corresponding image. We applied different mapping functions on sequence tensors and each function leads to a different pattern of image sequence. For instance, Figure 2 shows different sequence patterns by applying different mapping functions. Mapping functions are functions from elements in tensors to pixels in an image. The result is a matrix A where $A[i, j]$ shows the pixel value of image at position i and j . Our output images have the size of $224 \times 3 \times 3$. Our generated images have three colors per pixel (Red, Green, Blue). The following equations show the associated mapping functions for the generated images that assign colors to pixels.

Figure 2 (a) (i.e., Mapping 1):

$$A[i, j] = [((A[i, j, 0]) + (A[i, j, 1]) + (A[i, j, 2])) \% 256, (i * j) \% 256, (i + j) \% 256]$$

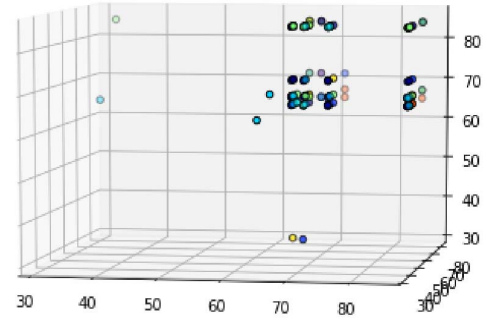
Figure 2 (b) (i.e., Mapping 2):

$$A[i, j] = [((A[i, j, 0]) \% 256 * (A[i, j, 1]) \% 256 * (A[i, j, 2]) \% 256),$$



(a) Clustering Images

Without connectivity constraints (time 0.02s)



(b) 3D Representation of Clustering

Fig. 1: Labeling tensors using agglomerative hierarchical clustering.

$$(i * j) \% 256, (i + 2 * j) \% 256]$$

Figure 2 (c) (i.e., Mapping 3):

$$A[i, j] = [(10 * (A[i, j, 0]) \% 256 * 20 * (A[i, j, 1]) \% 256 * (A[i, j, 2]) \% 256), (i * j) \% 256, (i + 2 * j) \% 256]$$

In this work, we used the sequence patterns represented in Figure 2 part (i) (i.e., mapping 9) which has the following mapping functions. The main rational behind choosing this specific mapping function is that we found this mapping function will lead to more accurate model for this problem.

$$A[i, j] = [i \% 256 + A[i, j, 0], j \% 256 + A[i, j, 1], (i + j) \% 256 + A[i, j, 2]]$$

4) Building dataset: train, test, and validation

In this step, we build and prepare the training, validation, and test image datasets required to feed as inputs to the convolutional neural network. Note that the obtained images have the size of $224 \times 224 \times 3$ and must be labeled which we made them labeled by using agglomerative hierarchical clustering. We need to have a training image dataset, a validation image

dataset, and a test dataset. Image dataset is constructed as we explained in the previous steps. We consider 72% of the total data set as the training data set and 12% of the total dataset as the validation dataset. Validation data set is needed to test how well our model performs against known labeled datasets. We also need a test dataset to test how well our model performs against unseen dataset. We consider 16% of the total data set as the test data set. To generate the test dataset we are using the following mapping function (which is similar to that of the training dataset):

$$A[i, j] = [i\%256 + A[i, j, 0], j\%256 + A[i, j, 1], (i + j)\%256 + A[i, j, 2]]$$

Figure 3 represents samples of the image patterns used for training, validation, and test dataset for this research.

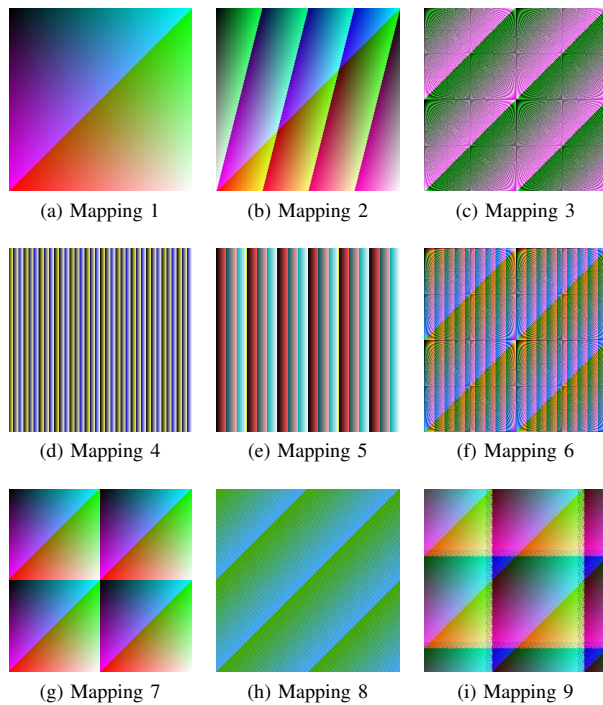


Fig. 2: Different mapping functions, different sequence images

5) Adapting convolutional neural network model to perform image classification

In this step, the generated image datasets are fed into the convolutional neural network. Figure 4 represents the CNN architecture that we used in this work (note that more details of each layer in our architecture are shown in Figure 7). As Figure 4 shows, there exists three types of layers: 1) convolutions 2) subsampling, and 3) fully connected layers. In this research, we build the model using Keras by leveraging the sequential model in Keras. We use the function `add(.)` in Keras to add layers to our model. Figure 7 shows the detailed architecture and filtering sizes that we used for our model. As the figure shows, our input images have the size of $224 \times 224 \times 3$. At the first step, we add the 1st convolutional

layer, then we add a max-pooling layer which results to images of the size of $112 \times 112 \times 25$. We keep adding more layers to the model, until the last layer where we add dropout. The dropout layer is used to avoid over-fitting of the model. The final layer, has 6 outputs which indicates the number of classes used to classify sequence images. The rational behind using this architecture is based on our experimental study in which we found that this architecture will lead to more accurate model for our problem.

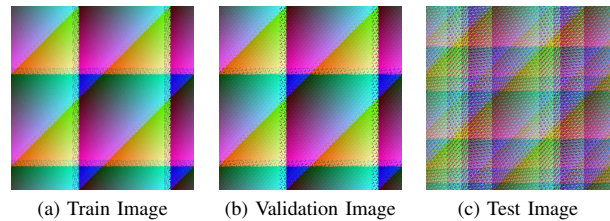


Fig. 3: Samples of train, test, and validation images

IV. EXPERIMENTAL RESULTS

We evaluated our Seq2Image model using human genome sequences. After transforming sequences to images and building the training, validation and testing datasets, we trained our model for 40 epochs. Each epoch runs over the entire training dataset. Out of 166 of $224 \times 224 \times 3$ sequence images, 120 images were used for training, 20 images were used for validation, and 26 images were used for testing. The obtained training accuracy using this model was 95.78. Figure 5 represents the training, validation accuracy and loss achieved over 40 epochs. As Figure 5 (a) indicates, the accuracy is achieving its max at 30th epochs and then the accuracy is remained almost the same. However, as shown in Figure 5 (b) the loss is decreasing when the number of epochs is increasing. While we achieved high training, validation, and testing accuracy, learning high-level features inside images needs a long time to train the network. To overcome this obstacle, a fine tuning approach was employed to make those high level features suitable for our training dataset¹.

TABLE II: Classification metrics.

Class	precision	recall	F1-score	support
0	1.00	1.00	1.00	32
1	1.00	0.91	0.95	32
2	1.00	0.88	0.93	16
3	0.80	1.00	0.89	20
4	1.00	1.00	1.00	9
5	1.00	1.00	1.00	11
accuracy			0.96	120
macro avg	0.97	0.96	0.96	120
weighted avg	0.97	0.96	0.96	120

We also evaluated classification metrics such as precision, recall, F1-score, and support for each class, and then built

¹ A GitHub repository at <https://github.com/NedaTavakoli/seq2Img> has been developed for this work, and can be granted upon request. Details related to fine tuning the model can be found in our repository.

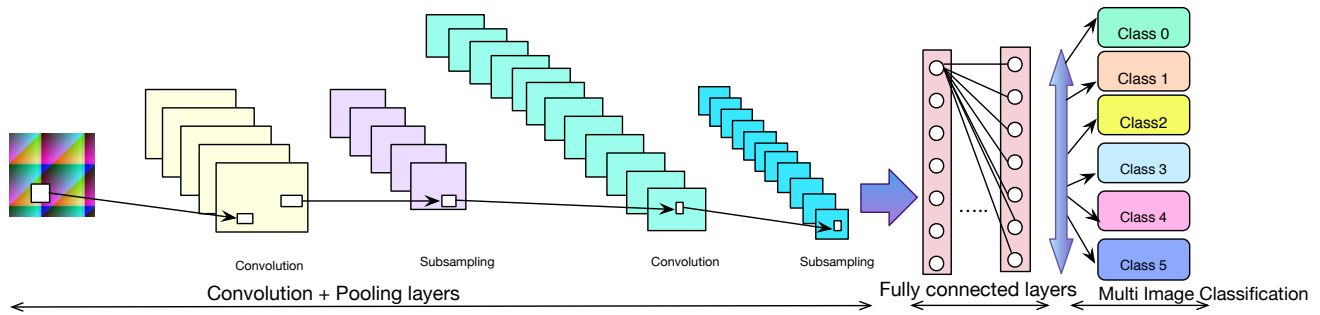
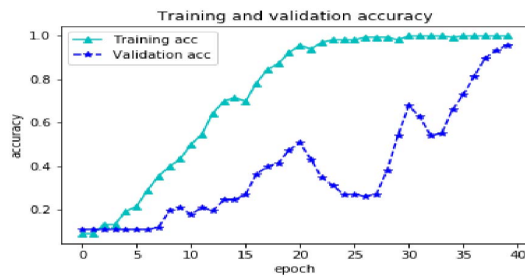
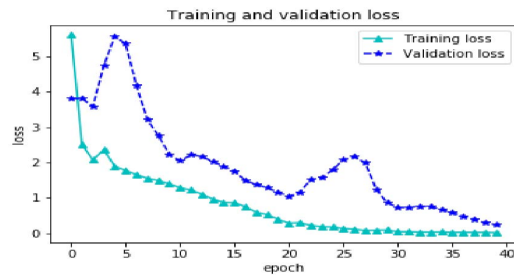


Fig. 4: The CNN architecture.



(a) Training and validation accuracy

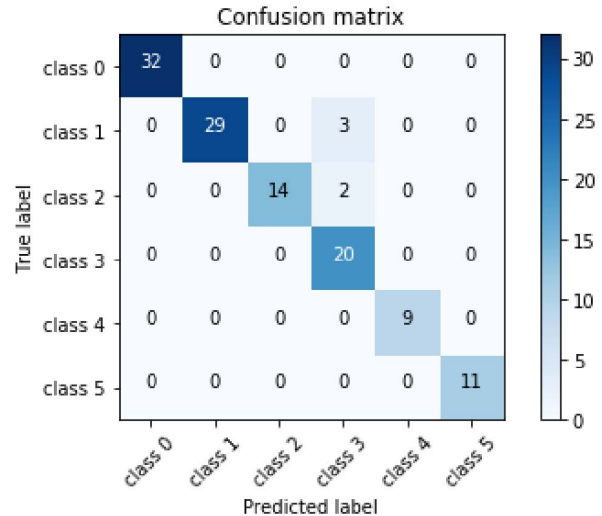


(b) Training and validation loss

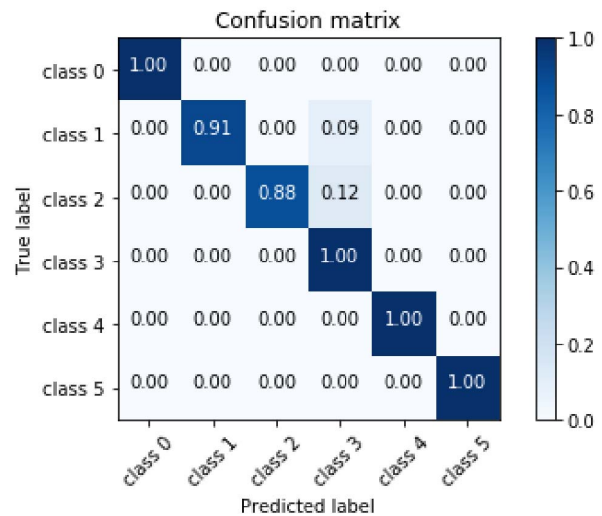
Fig. 5: Training and Validation accuracy and loss

a confusion matrix. Table II represents precision, recall, F1-score, and support metrics for each class that we obtained using sklearn classification metrics reports.

As the table shows, the overall precision/accuracy is 97%, which indicates how precise/accurate our model is out of total predicated positive images (i.e., true positive divided by total predicted positive). We obtained the overall recall of 96%, which indicates the ratio of the true positives to the all actual predicated positives (i.e., true positives over the total actual positives). F-1 score is used to measure a function precision and recall (i.e., $2 \times \text{precision} \times \text{recall}$ divided by $\text{precision} + \text{recall}$). As the table shows, we obtained the overall F-1 score of 96%, in fact the F1-score indicates the harmonic mean of precision and recall, we report the F1-score corresponding each class which represents the accuracy of our CNN classifier in classifying the images located in that particular class compared to the other classes. We also reported the support metric per class, which indicates the number of images with true labels that includes in each particular class.



(a) Confusion-matrix



(b) Normalized-confusion-matrix

Fig. 6: Confusion matrix.

We also generated the confusion matrix for our model. Confusion matrix or error matrix is used to visualize the performance of the model. Each row in the matrix represents

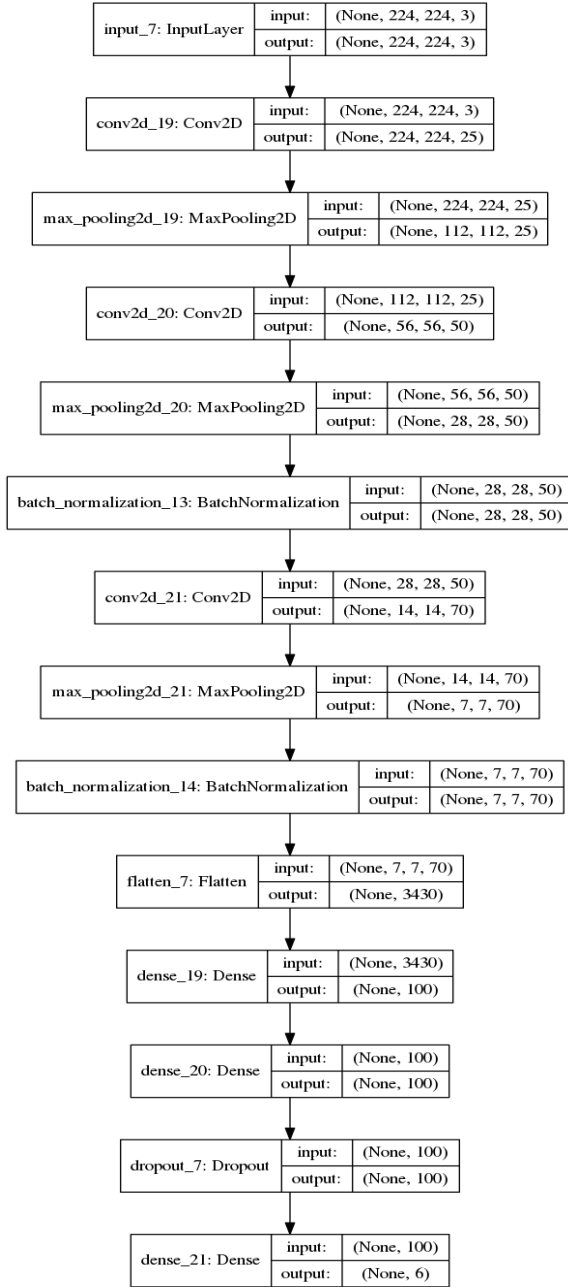


Fig. 7: Proposed seq2Image CNN architecture with near optimal parameter setting.

the number of instances in the predicted classes; however, each column shows the number of instances in the actual classes. The more diagonal values show the better confusion matrix and thus the better performance results. Figure 6 (a) shows the obtained confusion matrix of our model, and the Figure 6 (b) shows its normalized version.

V. CONCLUSION AND FUTURE WORK

In this work, we preformed a novel human genome classification by converting genome sequences to images and then using convolutional neural network to classify the obtained

images. In particular, we convert sequences to images, and then extract feature-vectors from images. The main challenging part of this problem is to extract important feature-vectors from sequence images. To tackle this problem we use a CNN model while fine tuning parameters in order to find the near optimal parameter setting for our model. Using this methods on our model, the overall training accuracy of 95.78%, 95.76% validation accuracy, and 95.83% testing accuracy are achieved which are very encouraging. As future works, we are planning to generalize our model to address subsequence matching problem as well as evaluating our model on other types of sequence datasets such as protein sequences.

REFERENCES

- [1] M. Deshpande and G. Karypis, "Evaluation of techniques for classifying biological sequences," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2002, pp. 417–431.
- [2] L. Wei and E. Keogh, "Semi-supervised time series classification," in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 748–753.
- [3] T. Lane and C. E. Brodley, "Temporal sequence learning and data reduction for anomaly detection," *ACM Transactions on Information and System Security (TISSEC)*, vol. 2, no. 3, pp. 295–331, 1999.
- [4] F. Sebastiani, "Machine learning in automated text categorization," *ACM computing surveys (CSUR)*, vol. 34, no. 1, pp. 1–47, 2002.
- [5] Z. Xing, J. Pei, and E. Keogh, "A brief survey on sequence classification," *ACM Sigkdd Explorations Newsletter*, vol. 12, no. 1, 2010.
- [6] "Genbank homepage," <http://www.ncbi.nlm.nih.gov/Genbank/>, accessed: 2010-09-30.
- [7] "Embl nucleotide sequence database homepage," <http://www.ebi.ac.uk/embl/>, accessed: 2010-09-30.
- [8] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "A comparison of arima and lstm in forecasting time series," in *IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 1394–1401.
- [9] N. Tavakoli, "Modeling genome data using bidirectional lstm," in *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, vol. 2. IEEE, 2019, pp. 183–188.
- [10] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "The performance of lstm and bilstm in forecasting time series," in *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2019, pp. 3285–3292.
- [11] N. Tavakoli, S. Siami-Namini, M. Adl Khanghah, F. Mirza Soltani, and A. Siami Namin, "An autoencoder-based deep learning approach for clustering time series data," *SN Applied Sciences*, vol. 2, pp. 1–25, 2020.
- [12] N. A. Chuzhanova, A. J. Jones, and S. Margetts, "Feature selection for genetic sequence classification," *Bioinformatics (Oxford, England)*, vol. 14, no. 2, pp. 139–143, 1998.
- [13] C. Leslie, E. Eskin, and W. S. Noble, "The spectrum kernel: A string kernel for svm protein classification," in *Biocomputing 2002*, 2001.
- [14] R. She, F. Chen, K. Wang, M. Ester, J. L. Gardy, and F. S. Brinkman, "Frequent-subsequence-based prediction of outer membrane proteins," in *Proceedings of the ninth ACM SIGKDD*, 2003, pp. 436–445.
- [15] S. Sonnenburg, G. Rätsch, and C. Schäfer, "Learning interpretable svms for biological sequence classification," in *Int. Conference on Research in Computational Molecular Biology*, 2005, pp. 389–407.
- [16] S. Sonnenburg, G. Rätsch, and B. Schölkopf, "Large scale genomic sequence svm classifiers," in *Proceedings of the 22nd international conference on Machine learning*, 2005, pp. 848–855.
- [17] R. M. Cormack, "A review of classification," *Journal of the Royal Statistical Society: Series A*, vol. 134, no. 3, pp. 321–353, 1971.